ENSF-381: Full Stack Web Development Laboratory

Ahmad Abdellatif
Department of Electrical & Software Engineering
University of Calgary

Lab 6

Objectives

Welcome to the ENSF381 course lab! In this lab, we will build on what we have learned in React by completing the React re-implementation of the User Directory and designing a Tic-Tac-Toe game. This lab will help reinforce key concepts like component structure, state management, and event handling while giving you hands-on experience with React's core principles.

Groups

Lab instructions must be followed in groups **of two students**.

Submission

- You must submit the complete source code, ensuring it can be executed without any modifications.
- If requested by the instructor, you may need to submit additional documentation (e.g., word documents and images).
- The link to the GitHub repository created for this lab must be provided alongside your submission on D2L. If you submit a word document as a part of this lab, include the link to the GitHub repository within that document as well.
- Only one member of the group needs to submit the assignment, but the submission must include the names and UCIDS of all group members.

Deadline

Lab exercises must be submitted by 11:55 PM on the same day as the lab session. Submissions made within 24 hours after the deadline will receive a maximum of 50% of the mark. Submissions made beyond 24 hours will not be evaluated and will receive a grade of zero.

Exercise 1: React User Directory

In this exercise, you will extend the React remake of the User Directory (Lab 05 Exercise 3). Last week, we focussed on the layout and component structure. This week, we will use props and hooks to implement the missing behavior.

The lab06 folder in the supplementary material already contains a React app with the overall layout, styling, user cards, and main component structure. Make a copy of the Starter folder and complete all your work for this lab in that copy.

- App.js renders the user directory and tic-tac-toe pages, depending on its state.
- exercise1/UserDirectoryPage.js houses the main component for Exercise 1.
- exercise1/Controls.js renders the delete and sorting controls for Exercise 1.
- exercise1/UserList.js renders the container for the cards.
- exercise1/UserCard.js renders a single user.
- exercise2/TicTacToePage.js contains the main component for Exercise 2.

API for Exercise 1

Use the same users_api endpoint you created for Lab 04:

Instructions

1. Open a terminal in the lab06 folder provided in the supplementary material.
2. Install the dependencies and start the development server:

```
npm install
npm start
```

3. In src/exercise1/UserDirectoryPage.js, import useState and useEffect from React. Then add state for the three pieces of data this page needs to manage:
    - users and setUsers: to store the list of users fetched from the API

- sortBy and setSortBy: to keep track of whether the current sort is by "id" or "group"
- viewMode and setViewMode: to keep track of whether the list is shown in "grid" or "list" view

4. In src/exercise1/UserDirectoryPage.js, use useEffect to fetch the initial list of users from the API when the component mounts, and update the users state variable with the fetched data using setUsers.

5. Once you have implemented the users and viewMode state in src/exercise1/UserDirectoryPage.js, update the rendered list so it uses those state values instead of the starter placeholders:

   - Change users={sampleUsers} to users={users}
   - Change viewMode="grid" to viewMode={viewMode}

6. In src/exercise1/Controls.js, keep the provided deleteId state for the delete input, and add the following missing event handlers:

   - Add an onChange handler on the delete <input> to update the deleteId state with setDeleteId.
   - Add an onClick handler on the Delete button that calls onDeleteClick(deleteId).
   - Add onClick handlers on the Sort by Group, Sort by ID, and Grid / List View buttons to call the corresponding props.

7. In src/exercise1/UserDirectoryPage.js, implement the provided handleSortByIdClick() function so it handles sorting by id. > Note: Do not sort the users state directly. Instead, create a new sorted array and update the state, using setUsers with that new array to ensure React detects the change and re-renders the component.

8. In src/exercise1/UserDirectoryPage.js, implement the provided handleSortByGroupClick() function so it handles sorting by group. Again, create a new sorted array and update the state using setUsers.

9. In src/exercise1/UserDirectoryPage.js, implement the provided handleViewToggleClick() function so it switches viewMode between "grid" and "list" using setViewMode.

10. Pass the implemented sort and view-toggle handler functions as props to the Controls component.

11. In src/exercise1/UserDirectoryPage.js, implement the provided handleDeleteClick(userId) function so it deletes a user by ID. The function should accept a user ID as an argument, make an API call to delete the user, and then use setUsers to update the state.

12. Pass the implemented delete handler as a prop to the Controls component.

13. In src/exercise1/UserList.js, update the list container so it uses viewMode to switch between the user-grid and user-list class names.

14. Test your implementation by interacting with the user directory page. You should be able to delete users, sort them by ID or group, and toggle between grid and list views.

Submission

1. Fill out the names and UCIDs of all group members in Answer_sheet.
2. Create a new GitHub repository and upload the code for your completed lab exercise to both the new repository and D2L.

NOTE: Do not include the node_modules folder

Exercise 2: Tic-Tac-Toe

This exercise has been modified for this course based on the Tic-Tac-Toe tutorial.

Open the Tic-Tac-Toe Page

- Open the same Lab 06 app from Exercise 1.
- Use the page-switch buttons in App.js to open the **Exercise 2: Tic-Tac-Toe** page.
- Open src/exercise2/TicTacToePage.js in your IDE.

Understanding the Starter Code

Now open the TicTacToePage component (src/exercise2/TicTacToePage.js) in your project folder. This file establishes the foundation for the tic-tac-toe game and includes the necessary CSS styles, allowing us to focus on learning React and building the game's functionality.

The code will be structured around three main components:

- Square – A React component that renders a single <button>.
- Board – A React component that displays a 3x3 grid of 9 squares.
- calculateWinner – A function that contains the game's logic for determining the winner based on the current state of the squares. This function evaluates the state of the board and returns the winner, if any, based on the updated logic. It will also helps us keep track of the game's state and render the board with the correct values.

Note: All the three components need to be placed in TicTacToePage.js.

Building the Board

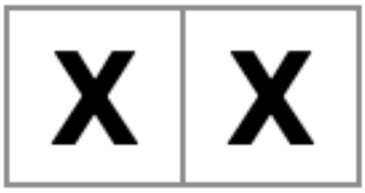
Open src/exercise2/TicTacToePage.js and create a function called Square as shown below.

```
function Square() {
  return (
    <div>
      <button className="square">X</button>
    </div>
  );
}
```

At the moment, the board contains just one square, but we need nine. If you try duplicating the square to create two, your code might look something like this:

```
function Square() {
  return (
    <>
      <button className="square">X</button>
```

```
    <button className="square">X</button>
  </>
 );
}
```



To create a full 3x3 grid, duplicate the square code a few more times. However, you will notice that the squares are arranged in a single line rather than a grid.

To fix this, we will group the squares into rows using `<div>` elements and apply CSS styles. Adding numbers to each square will also help keep track of their positions.

Copy the following styles into `src/App.css`:

```
.ttt-board {
  width: fit-content;
}

.board-row {
  display: flex;
}

.square {
  width: 100px;
  height: 100px;
```

```
  border: 1px solid #162033;
  background: #ffffff;
  font-size: 24px;
  font-weight: 700;
  cursor: pointer;
}
```

In TicTacToePage.js, update the Square component like this:

```
function Square() {
  return (
    <>
      <div className="board-row">
        <button className="square">1</button>
        <button className="square">2</button>
        <button className="square">3</button>
      </div>
      <div className="board-row">
        <button className="square">4</button>
        <button className="square">5</button>
        <button className="square">6</button>
      </div>
      <div className="board-row">
        <button className="square">7</button>
        <button className="square">8</button>
        <button className="square">9</button>
      </div>
    </>
  );
}
```

Now you should see:

1	2	3
4	5	6
7	8	9

Passing Data Through Props

Next, we want each square to show an "X" when clicked. Instead of writing the same code for each of the 9 squares, we can use React's reusable component system. This enables us to create a single Square component that can be used for all squares, making our code simpler and more efficient.

Start by creating a new function named Board to manage the grid layout. This will help organize our code and make it easier to update the board as we add more functionality.

```
function Board() {
  //...
}
```

After creating Board, render it inside TicTacToePage:

```
function TicTacToePage() {
  return (
    <>
      <section className="panel">
        <h1>Tic-Tac-Toe</h1>
```

```
    </section>

    <section className="panel">
      <div className="ttt-board">
        <Board />
      </div>
    </section>
  </>
 );
}
```

Next, we will move the square rendering logic into the Board component. After that, update the Board component to pass a value (i.e., the number) as a prop to each Square. Finally, modify the Square component to accept and handle this prop accordingly:

```
function Square({ value }) {
 return <button className="square">{value}</button>;
}
```

Update the Board component to pass values to Square:

```
function Board() {
 return (
  <>
    <div className="board-row">
      <Square value="1" />
      <Square value="2" />
      <Square value="3" />
    </div>
    <div className="board-row">
      <Square value="4" />
      <Square value="5" />
      <Square value="6" />
```

```
    </div>
    <div className="board-row">
      <Square value="7" />
      <Square value="8" />
      <Square value="9" />
    </div>
  </>
 );
}
```

Now you should see a grid of numbers:

1	2	3
4	5	6
7	8	9

tic-tac-toe board filled with numbers 1 through 9

Making the Squares Interactive

Now, let us enable each square to display an "X" when clicked.

Handling Click Events

Start by adding an event handler inside the Square component:

```
function Square({ value }) {
  function handleClick() {
    console.log("clicked!");
  }

  return (
    <button className="square" onClick={handleClick}>
      {value}
    </button>
  );
}
```

Now, when you click a square, "clicked!" appears in the console. If you click multiple times, the browser, depending on the browser, might not log duplicate lines but will instead display a counter next to the message.

Storing Clicked State

Now, we want each square to “remember” when it has been clicked and update the text it displays to "X". In React, components use state to store information that changes over time. We will use useState for this.

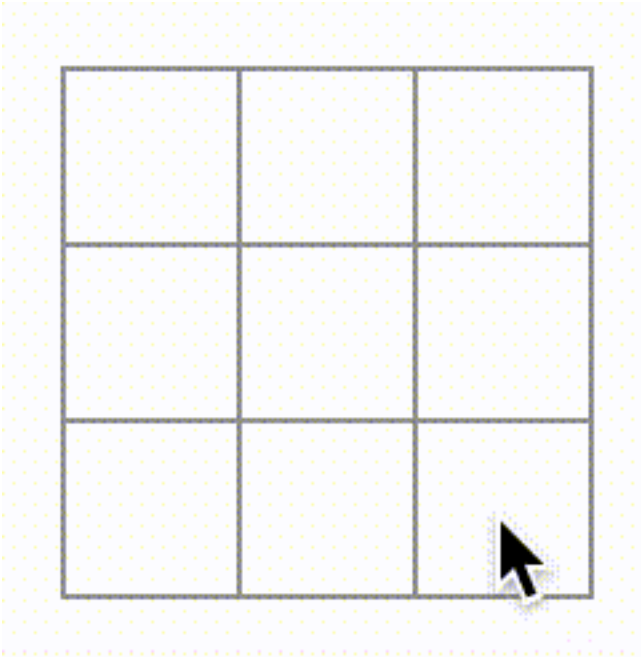
Step 1: Import useState at the top of the file

```
import { useState } from "react";
```

Step 2: Remove the value prop from Square and initialize state

```
function Square() {
  const [value, setValue] = useState(null);
}
```

Now, each square will handle its own state and update accordingly when clicked.



adding xes to board

Since the Square component no longer accepts props, we need to remove the value prop from all nine of the Square components created by the Board component:

```
// ...

function Board() {
  return (
    <>
      <div className="board-row">
        <Square />
        <Square />
        <Square />
      </div>
      <div className="board-row">
        <Square />
        <Square />
        <Square />
      </div>
```

```
    <div className="board-row">
      <Square />
      <Square />
      <Square />
    </div>
  </>
 );
}
```

Updating State on Click

Now, let us update the Square component so that clicking it changes its value to "X".

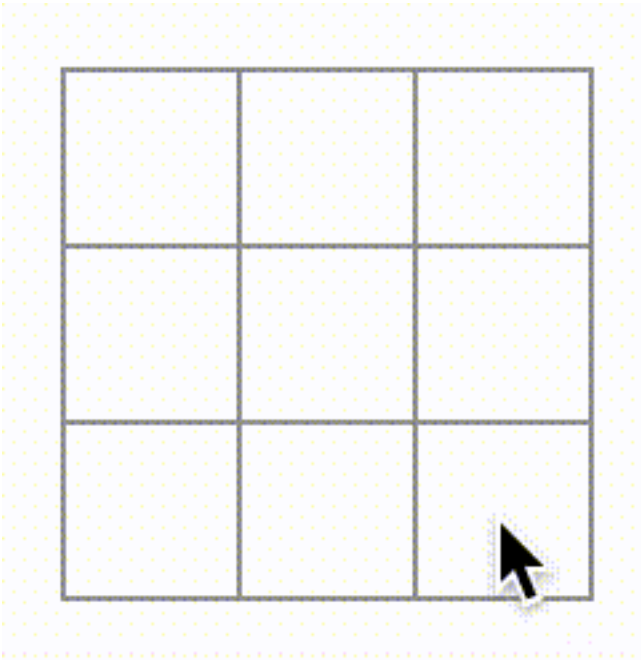
Replace console.log("clicked!") with setValue("X"):

```
function Square() {
 const [value, setValue] = useState(null);

 function handleClick() {
  setValue("X");
 }

 return (
  <button className="square" onClick={handleClick}>
   {value}
  </button>
 );
}
```

By calling setValue("X") inside handleClick, we tell React to update the state and re-render the Square when it's clicked. After this change, clicking a square will display "X".

Independent State for Each Square

Each Square maintains its own state, so clicking one square doesn't affect the others. When a component's (in this case, Square) state updates, React automatically re-renders that component along with any necessary child components.



adding xes to board

Completing the Game

With the Tic-Tac-Toe board set up, it is time to add interactivity. The next steps include:

- Alternating turns between "X" and "O" when clicking squares.
- Implementing logic to determine the winner.

Managing State in the Board Component

To keep track of the game state, we need to update the Board component to store the state of all 9 squares. We will use a state variable called squares, initialized as an array of nine null values, indicating that all squares are empty at the start.

```
// ...

function Board() {
  const [squares, setSquares] = useState(Array(9).fill(null));

  return (
    // ...
  );
}
```

The expression Array(9).fill(null) creates an array with nine elements, all set to null. The useState hook initializes the squares state variable with this array. Each element in the array represents a square on the board.

As players take turns, the squares array will be updated to reflect their moves. For example:

```
['O', null, 'X', 'X', 'X', 'O', 'O', null, null]
```

Passing State to the Squares

Now, each Square will receive its value as a prop from the Board component:

```
<Square value={squares[i]} />
```

Each Square now gets a value prop, which can be 'X', 'O', or null (for the empty squares). To reflect this change, update the Square component to receive the value prop from Board. Since the Board component manages state, we can remove the internal state from Square **(remove handleClick function and useState from Square function):**

```
function Square({ value }) {
  return <button className="square">{value}</button>;
}
```

At this stage, the Tic-Tac-Toe board should be rendered, but the squares will not be interactive yet. To allow players to interact with the board, we need to update the state when a square is clicked. However, since Board owns the state, Square cannot update it directly.

To solve this, we will pass a function from Board to Square. When a square is clicked, it will call this function to notify Board about the interaction. The function we pass will call setSquares, which is necessary to make changes to the Board's state.

Handling Click Events

Start by creating a function, called onSquareClick, in Board that Square can call when clicked:

```
function Square({ value, onSquareClick }) {
  return (
    <button className="square" onClick={onSquareClick}>
      {value}
    </button>
  );
}
```

Next, connect onSquareClick to an actual function inside Board that updates the game state. To do this, pass a function to the onSquareClick prop of each Square component:

```
function Board() {
  const [squares, setSquares] = useState(Array(9).fill(null));

  return (
    <>
      <div className="board-row">
        <Square value={squares[0]} onSquareClick={handleClick} />
        {/* ... */}
      </div>
    </>
  );
}
```

Finally, define the handleClick function inside Board. This function will modify the squares array and update the board's state using setSquares:

```
function Board() {
  const [squares, setSquares] = useState(Array(9).fill(null));

  function handleClick() {
    const nextSquares = squares.slice();
    nextSquares[0] = "X";
    setSquares(nextSquares);
  }

  return (
    // ...
  );
}
```

The handleClick function first creates a copy of the squares array using slice(). It then updates the value of the first square (index 0) to 'X'. Finally, calling setSquares notifies React to re-render the Board and its child components, including Square.

Why does this work?

JavaScript closures allow handleClick to access both the squares state and the setSquares function because they are defined within the same scope.

At this point, clicking the upper-left square will display an 'X'. However, the function is hardcoded to update only the first square. Let's generalize it so that any square can be updated.

Updating Any Square

Modify handleClick to accept an argument i, representing the square's index:

```
function handleClick(i) {
  const nextSquares = squares.slice();
  nextSquares[i] = "X";
  setSquares(nextSquares);
}
```

Now, pass the correct index when rendering each Square. You might try this:

```
<Square onClick={handleClick(0)} />
```

However, this will not work. The function will execute immediately during rendering, causing an infinite loop.

To prevent this, wrap handleClick(i) in an arrow function so it runs only when clicked:

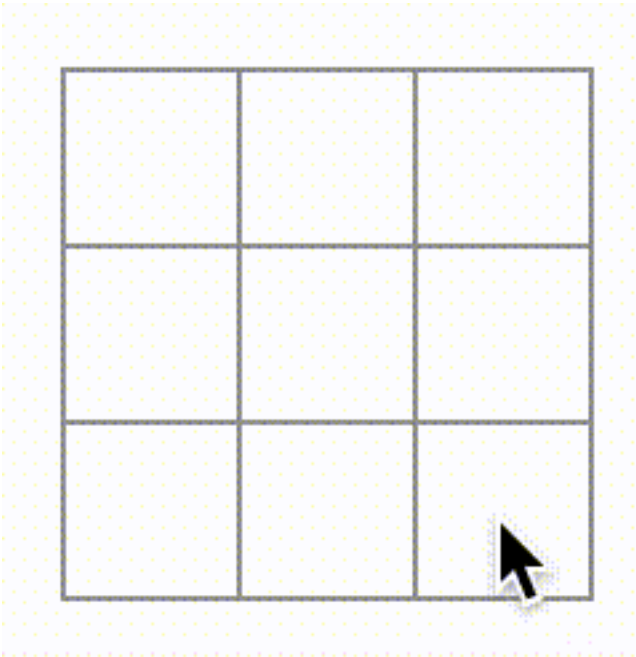
```
<Square value={squares[0]} onSquareClick={() => handleClick(0)} />
```

Now, update all Square components to pass the correct index:

```
function Board() {
  // ...

  return (
    <>
      <div className="board-row">
        <Square value={squares[0]} onSquareClick={() => handleClick(0)} />
        <Square value={squares[1]} onSquareClick={() => handleClick(1)} />
        <Square value={squares[2]} onSquareClick={() => handleClick(2)} />
      </div>
      <div className="board-row">
        <Square value={squares[3]} onSquareClick={() => handleClick(3)} />
        <Square value={squares[4]} onSquareClick={() => handleClick(4)} />
        <Square value={squares[5]} onSquareClick={() => handleClick(5)} />
      </div>
      <div className="board-row">
        <Square value={squares[6]} onSquareClick={() => handleClick(6)} />
        <Square value={squares[7]} onSquareClick={() => handleClick(7)} />
        <Square value={squares[8]} onSquareClick={() => handleClick(8)} />
      </div>
    </>
```

```
  );
}
```

Now you can again add X's to any square on the board by clicking on them:



filling the board with X

Now that the Board component is in charge of state, it tells each Square what to display. When you click a Square, it doesn't change itself—instead, it asks the Board to update the game state. React handles the rest, re-rendering the Board and all Squares automatically. Keeping the game state inside the Board also makes it easier to check for a winner later.

What Happens When You Click a Square?

- Clicking a Square calls the onClick function it got from the Board.
- That triggers handleClick in the Board, passing the square's index.
- The function updates the squares array, replacing null with 'X'.
- The Board's state changes, causing React to re-render the Board and all Squares.
- The clicked Square now shows 'X' instead of being empty.

Taking Turns

Right now, only 'X' appears. To alternate turns, in Board function add another state variable:

```
function Board() {
  const [xIsNext, setXIsNext] = useState(true);
  const [squares, setSquares] = useState(Array(9).fill(null));
}
```

Each turn, xIsNext flips between true and false, deciding whether the next move should be 'X' or 'O'. Update handleClick like this:

```
function handleClick(i) {
  const nextSquares = squares.slice();

  if (xIsNext) {
    nextSquares[i] = 'X';
  } else {
    nextSquares[i] = 'O';
  }

  setSquares(nextSquares);
  setXIsNext(!xIsNext);
}
```

Now, clicks alternate between 'X' and 'O', making the game playable.

Fixing the Overwrite Issue

There is a bug: clicking the same square twice replaces 'X' with 'O'. To prevent that, stop the function from updating a square if it is already filled:

```
function handleClick(i) {
  if (squares[i]) return;
```

```
  const nextSquares = squares.slice();
}
```

With this fix, once a square is marked, it stays that way just like in a real game.

Declaring a Winner

Now that players can take turns, we need to implement logic to check for a winner and prevent further moves once the game is won. This can be achieved by adding a helper function called calculateWinner, which checks the state of all 9 squares, determines if there's a winner, and returns either 'X', 'O', or null (if no winner).

Here's the function that checks for the winner:

```
function calculateWinner(squares) {
  const lines = [
    // Horizontal rows
    [0, 1, 2],
    [3, 4, 5],
    [6, 7, 8],
    // Vertical columns
    [0, 3, 6],
    [1, 4, 7],
    [2, 5, 8],
    // Diagonal lines
    [0, 4, 8],
    [2, 4, 6],
  ];

  // Check each line for a winner
  for (let i = 0; i < lines.length; i++) {
    const [a, b, c] = lines[i];

    if (squares[a] && squares[a] === squares[b] && squares[a] === squares[c]) {
```

```
    return squares[a]; // Return the winning symbol ('X' or 'O')
  }
 }

 return null; // No winner
}
```

Key Details:

- **Lines Array**: The lines array stores the index positions for all possible winning combinations (horizontal, vertical, and diagonal).
- **Logic**: The function checks if all three squares in any of the winning lines are filled with the same symbol ('X' or 'O'). If a winner is found, it returns that symbol; otherwise, it returns null.

Next, call calculateWinner(squares) within the handleClick function in the Board component to check if a player has won. This check should happen alongside verifying if the clicked square is already filled. If either condition is true, the function should return early:

```
function handleClick(i) {
 if (calculateWinner(squares) || squares[i]) {
  return;
 }

 const nextSquares = squares.slice();

 //...
}
```

To inform players when the game has ended, a status message should be displayed. If there is a winner, the message should indicate who won. If the game is still in progress, it should show whose turn it is:

```
function Board() {
  // ...

  const winner = calculateWinner(squares);
  let status;

  if (winner) {
    status = 'Winner: ' + winner;
  } else {
    status = 'Next player: ' + (xIsNext ? 'X' : 'O');
  }

  return (
    <>
      <div>{status}</div>
      <div className="board-row">
        // ...
      </div>
    </>
  );
}
```

Final Touches

Once you integrate these updates into your Board component, you will have a fully functional Tic-Tac-Toe game that displays the winner when the game ends and tracks whose turn it is. You have also just learned how to implement basic React functionality such as state updates, event handling, and rendering dynamic content.

Submission

1. Fill out the names and UCIDs of all group members in Answer_sheet.

2. Add your completed Lab 06 code, including both exercises, to the GitHub repository you created for this lab. Then upload the same project to D2L as a single compressed file so it can be executed without modification. **Do not include the node_modules folder.**